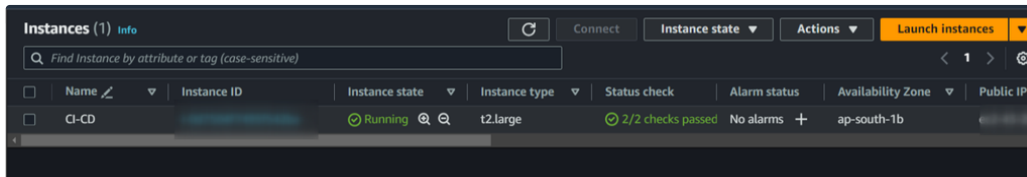


ZomatoClone: Deployment with DevSecOps CI/CD

Step By Step Implementation: [🔗](#)

STEP1:Launch an Ubuntu(22.04) T2 Large Instance [🔗](#)

Launch an AWS T2 Large Instance. Use the image as Ubuntu. You can create a new key pair or use an existing one. Enable HTTP and HTTPS settings in the Security Group and open all ports (not best case to open all ports but just for learning purposes it's okay).



Step 2 — Install Jenkins, Docker and Trivy [🔗](#)

To Install Jenkins [🔗](#)

Connect to your console, and enter these commands to Install Jenkins

```
1 vim jenkins.sh
```

Copy and paste below script in it :

```
1 #!/bin/bash
2 sudo apt update -y
3 #sudo apt upgrade -y
4 wget -O - https://packages.adoptium.net/artifactory/api/gpg/key/public | tee /etc/apt/keyrings/adoptium.asc
5 echo "deb [signed-by=/etc/apt/keyrings/adoptium.asc] https://packages.adoptium.net/artifactory/deb $(awk -F=
'^VERSION_CODENAME/{print$2}' /etc/os-release) main" | tee /etc/apt/sources.list.d/adoptium.list
6 sudo apt update -y
7 sudo apt install temurin-17-jdk -y
8 /usr/bin/java --version
9 curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo tee \
10 /usr/share/keyrings/jenkins-keyring.asc > /dev/null
11 echo deb [signed-by=/usr/share/keyrings/jenkins-keyring.asc] \
12 https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
13 /etc/apt/sources.list.d/jenkins.list > /dev/null
14 sudo apt-get update -y
15 sudo apt-get install jenkins -y
16 sudo systemctl start jenkins
17 sudo systemctl status jenkins
```

Execute the Script:

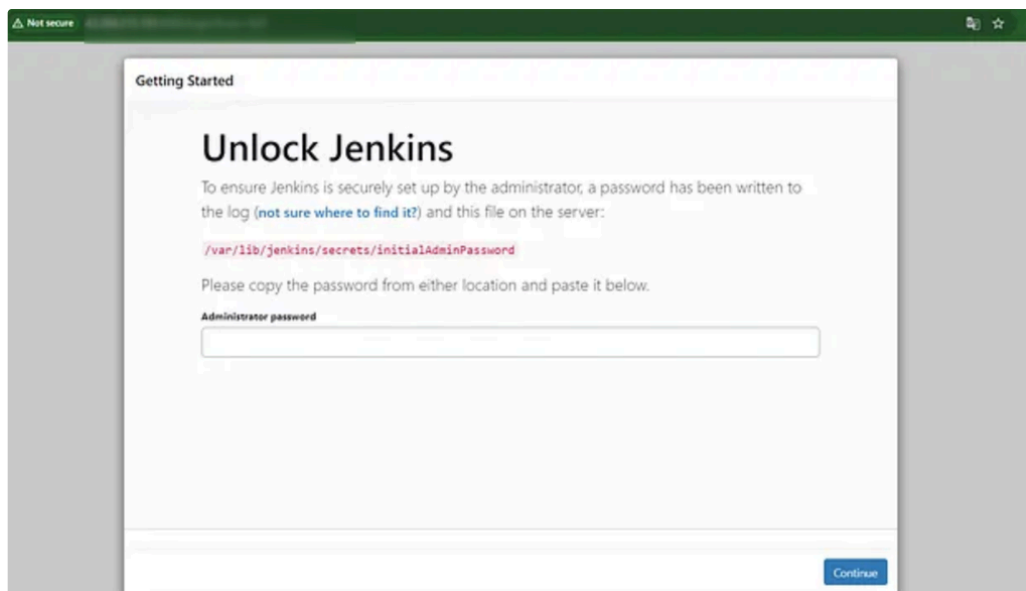
```
1 sudo chmod 777 jenkins.sh
2 ./jenkins.sh # this will install jenkins
```

Once Jenkins is installed, you will need to go to your AWS EC2 Security Group and open Inbound Port 8080, since Jenkins works on Port 8080.

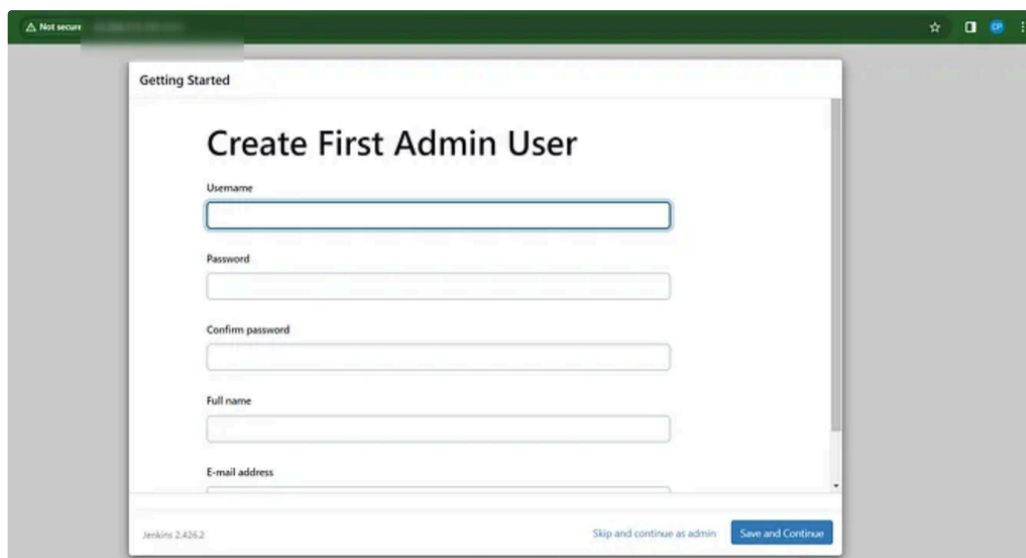
Now, grab your Public IP Address

```
1 <EC2 Public IP Address:8080>
```

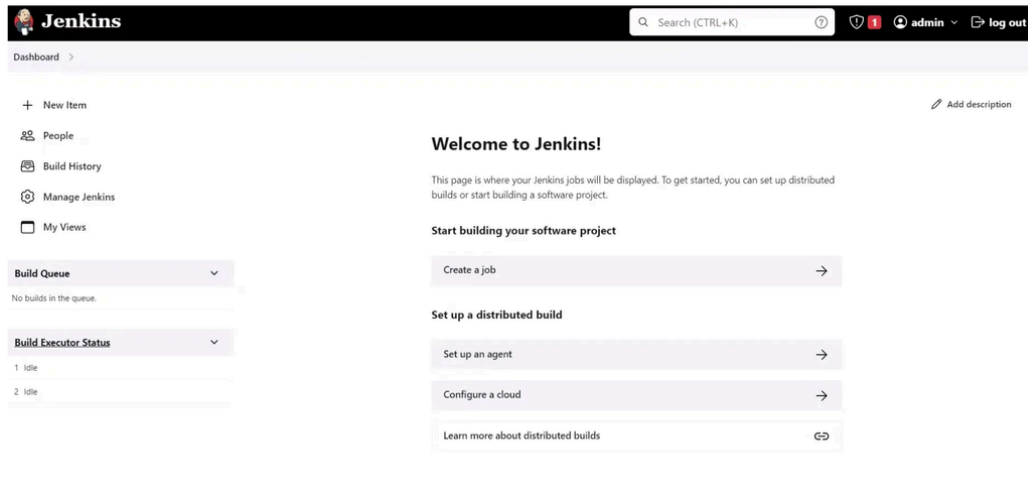
```
2 sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```



Unlock Jenkins using an administrative password and install the suggested plugins.



Jenkins will now get installed and install all the libraries.



Create a user click on save and continue.

Jenkins Getting Started Screen.

Install Docker: [🔗](#)

create script , as we have done in Jenkins installation.

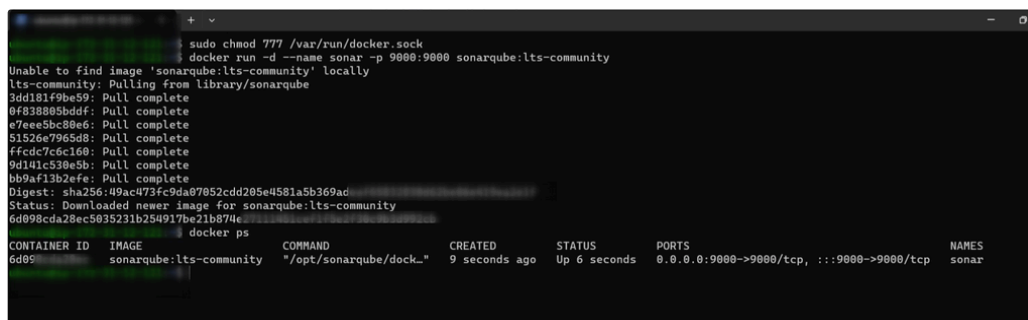
```
1 sudo apt-get update
2 sudo apt-get install docker.io -y
3 sudo usermod -aG docker $USER #my case is ubuntu
4 newgrp docker
5 sudo chmod 777 /var/run/docker.sock
```

execute it :

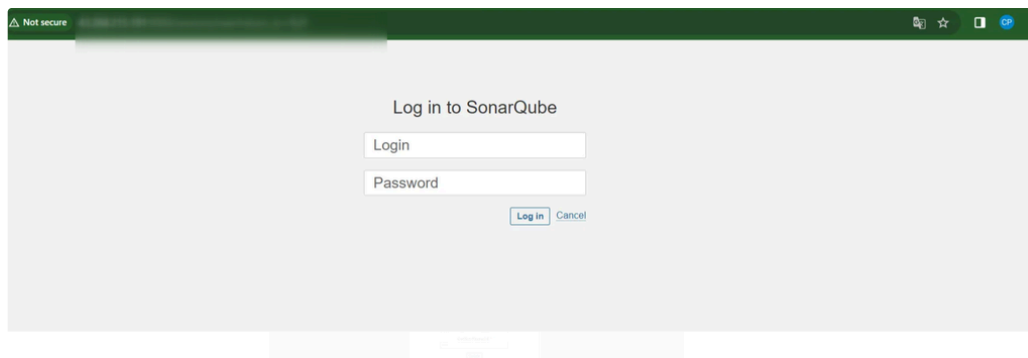
```
1 chmod +x docker.sh
2 ./docker.sh
```

After the docker installation, we create a sonarqube container (Remember to add 9000 ports in the security group).

```
1 docker run -d --name sonar -p 9000:9000 sonarqube:lts-community
```

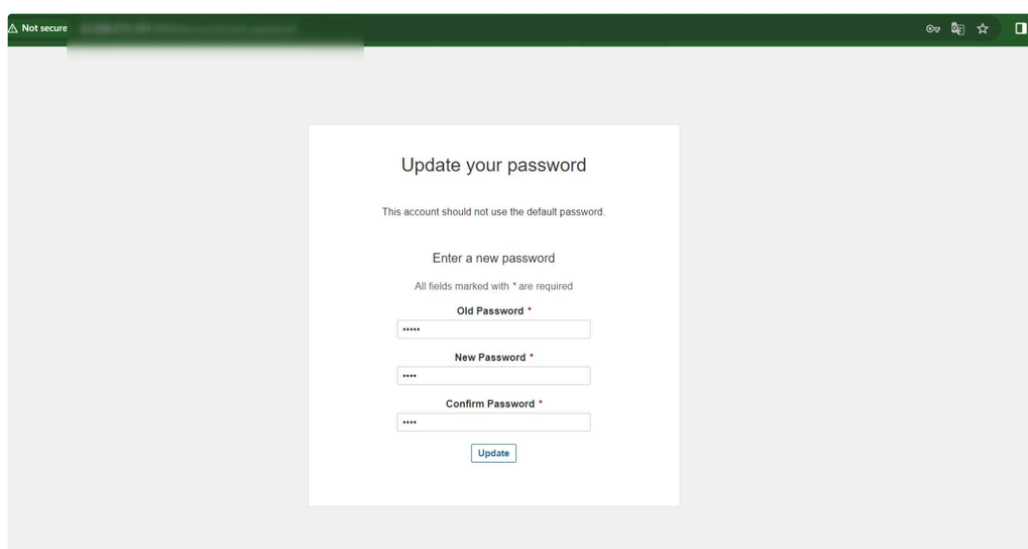


Now our sonarqube is up and running

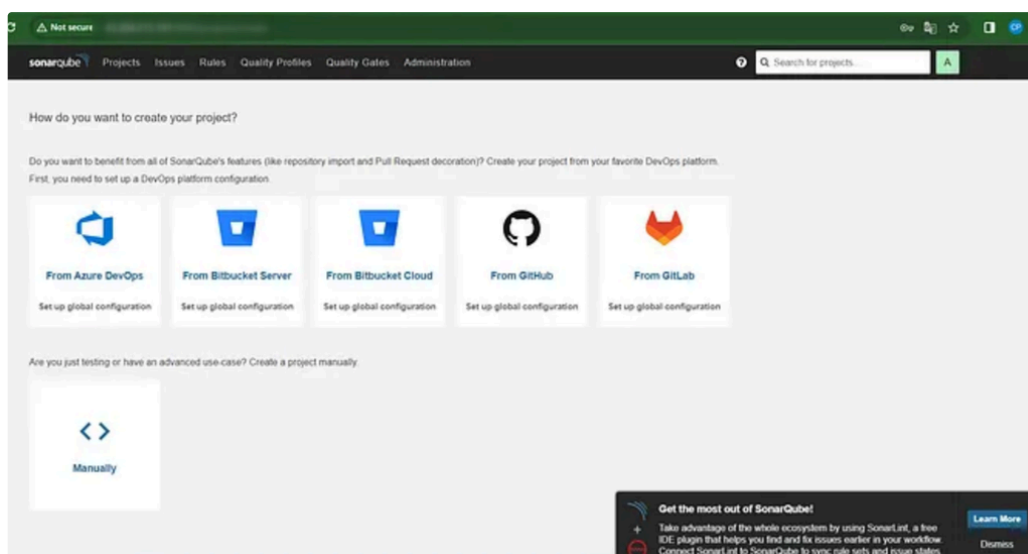


Enter username and password, click on login and change password

```
username admin
password admin
```



Update New password, This is Sonar Dashboard.



Install Trivy [🔗](#)

Create Script

```
1 vim trivy.sh
```

Copt paste the code in script.

```
1 sudo apt-get install wget apt-transport-https gnupg lsb-release -y
2 wget -qO - https://aquasecurity.github.io/trivy-repo/deb/public.key | gpg --dearmor | sudo tee
  /usr/share/keyrings/trivy.gpg > /dev/null
3 echo "deb [signed-by=/usr/share/keyrings/trivy.gpg] https://aquasecurity.github.io/trivy-repo/deb $(lsb_release
  -sc) main" | sudo tee -a /etc/apt/sources.list.d/trivy.list
4 sudo apt-get update
5 sudo apt-get install trivy -y
```

execute script

```
1 chmod +x trivy.sh
2 ./trivy.sh
```

Next, we will log in to Jenkins and start to configure our Pipeline in Jenkins

Step 3 — Install Plugins like JDK, Sonarqube Scanner, NodeJs, OWASP Dependency Check [↗](#)

Install Plugin [↗](#)

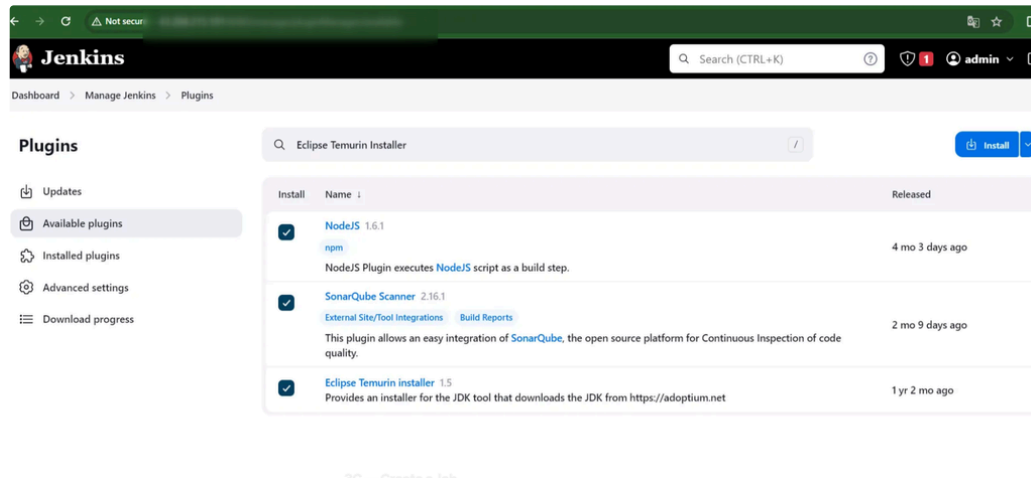
Go to Manage Jenkins → Plugins → Available Plugins →

Install below plugins

1 → Eclipse Temurin Installer (Install without restart)

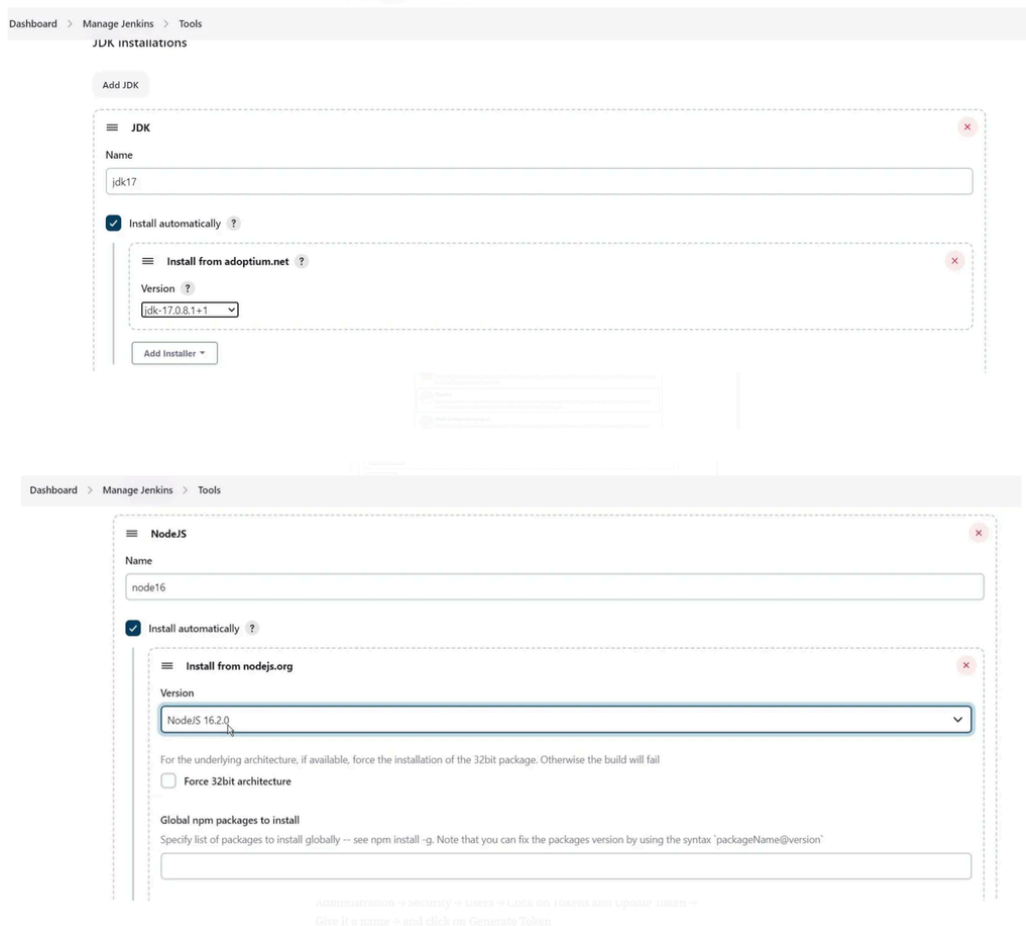
2 → SonarQube Scanner (Install without restart)

3 → NodeJs Plugin (Install Without restart)



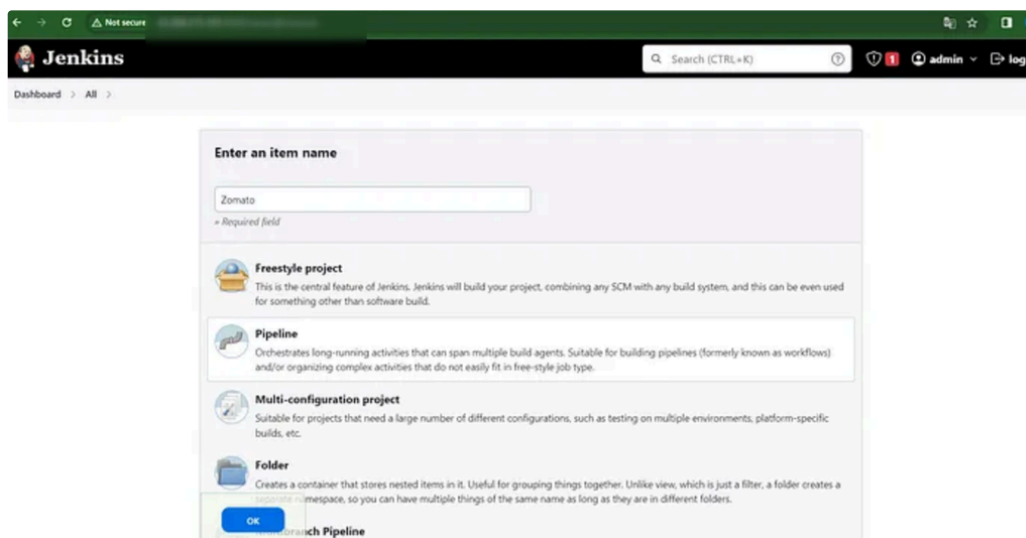
Configure Java and Nodejs in Global Tool Configuration [↗](#)

Goto Manage Jenkins → Tools → Install JDK(17) and NodeJs(16) → Click on Apply and Save



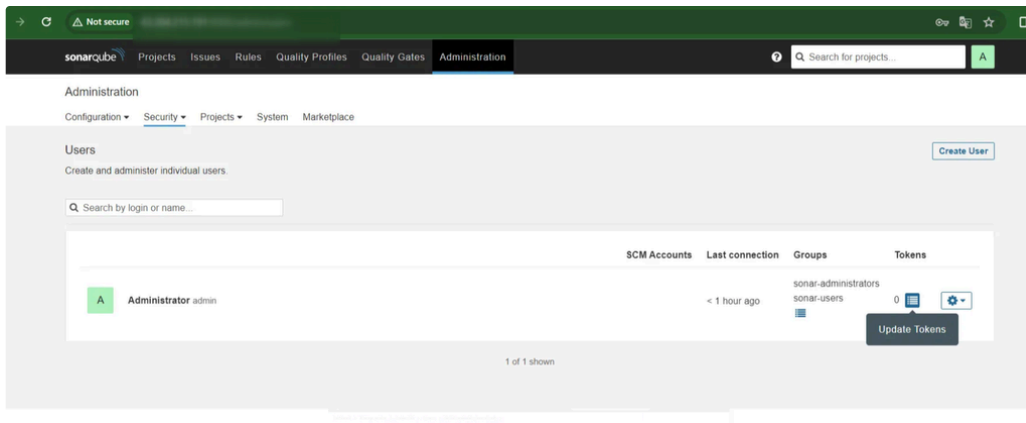
Create a Job [🔗](#)

Create a job as Zomato Name, select pipeline and click on ok.



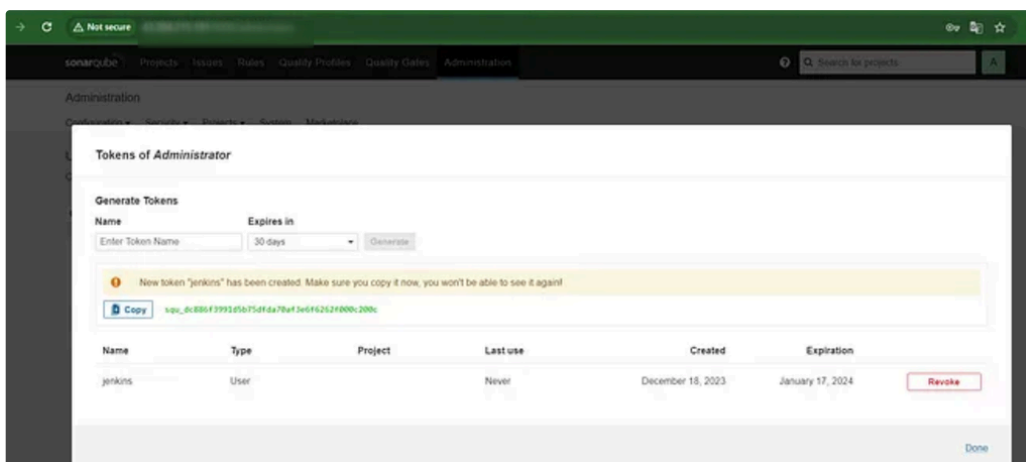
Step 4 — Configure Sonar Server in Manage Jenkins [🔗](#)

Grab the Public IP Address of your EC2 Instance, Sonarqube works on Port 9000, so <Public IP>:9000. Goto your Sonarqube Server. Click on Administration → Security → Users → Click on Tokens and Update Token → Give it a name → and click on Generate Token



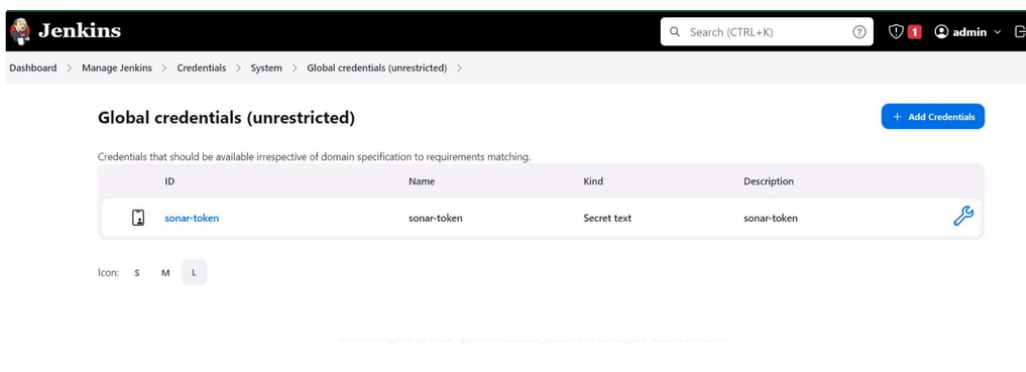
click on update Token

Create a token with a name and generate



copy Token and keep it safe.

Goto Jenkins Dashboard → Manage Jenkins → Credentials → Add Secret Text. It should look like this



You will this page once you click on create

Dashboard > Manage Jenkins > System >

SonarQube servers

If checked, job administrators will be able to inject a SonarQube server configuration as environment variables in the build.

☐ Environment variables

SonarQube installations

List of SonarQube installations

Name

sonar-server

Server URL

Default is http://localhost:9000

http://172.31.12.121:9000

Server authentication token

SonarQube authentication token. Mandatory when anonymous access is disabled.

sonar-token

+ Add

Save Apply

Now, go to Dashboard → Manage Jenkins → System and Add like the below image.

Dashboard > Manage Jenkins > Tools

SonarQube Scanner installations

Add SonarQube Scanner

SonarQube Scanner

Name

sonar-scanner

☒ Install automatically ?

Install from Maven Central

Version

SonarQube Scanner 5.0.1.3006

Add Installer

Add SonarQube Scanner

Save Apply

Click on Apply and Save

The Configure System option is used in Jenkins to configure different server

Global Tool Configuration is used to configure different tools that we install using Plugins

We will install a sonar scanner in the tools.

Jenkins Search (CTRL+K) admin log out

Dashboard > Zomato >

Status

Changes

Build Now

Configure

Delete Pipeline

Full Stage View

SonarQube

Rename

Pipeline Syntax

Build History trend

Filter builds...

Dec 18, 2023, 1:58 PM

Atom feed for all Atom feed for failures

Stage View

	Declarative: Tool Install	clean workspace	Checkout from Git	Sonarqube Analysis	quality gate	Install Dependencies
Average stage times: (Average full run time: ~1min 57s)	31s	375ms	5s	27s	650ms	45s
Dec 18 1928	31s	375ms	5s	27s	650ms (skipped for 188ms)	45s

SonarQube Quality Gate

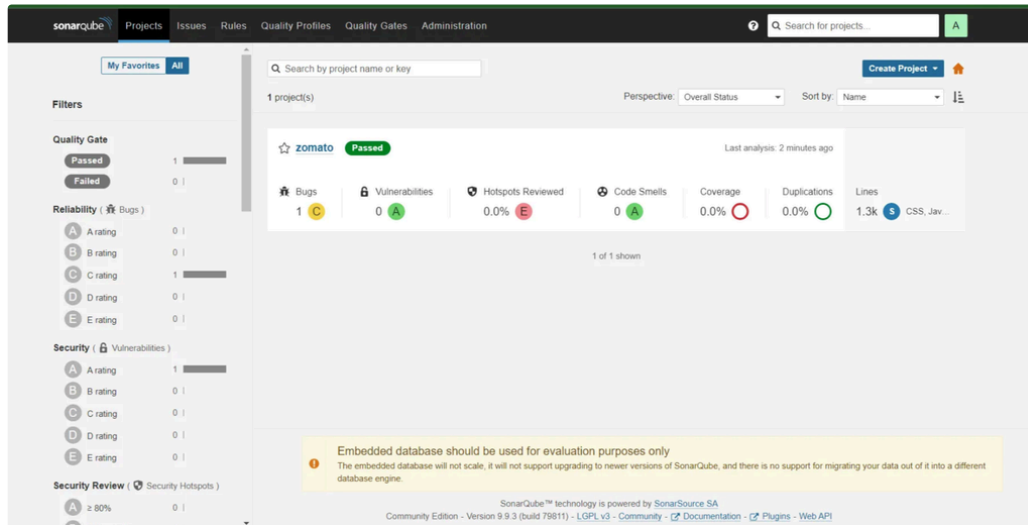
zomato **Passed**

server-side processing: **Success**

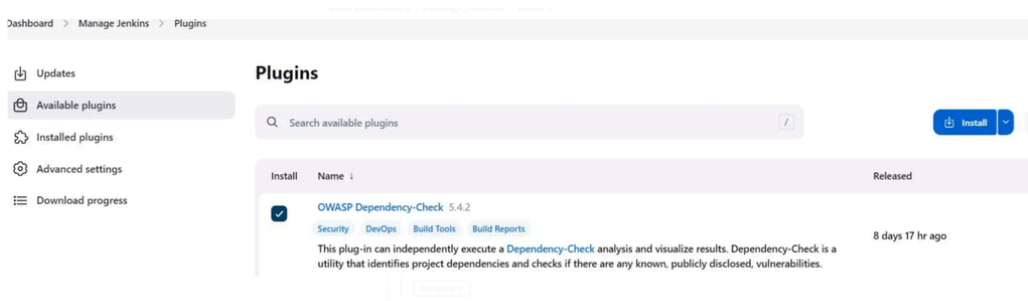
Permalinks

In the Sonarqube Dashboard add a quality gate also

Administration → Configuration → Webhooks



Click on Create



Add details

```
1 #in url section of quality gate
2 <http://jenkins-public-ip:8080>/sonarqube-webhook/
```

Dependency-Check installations

Add Dependency-Check

Dependency-Check

Name

DP-Check

☒ Install automatically ?

Install from github.com

Version

dependency-check 6.5.1

Add Installer ▾

Let's go to our Pipeline and add the script in our Pipeline Script.

```

1 pipeline{
2   agent any
3   tools{
4     jdk 'jdk17'
5     nodejs 'node16'
6   }
7   environment {
8     SCANNER_HOME=tool 'sonar-scanner'
9   }
10  stages {
11    stage('clean workspace'){
12      steps{
13        cleanWs()
14      }
15    }
16    stage('Checkout from Git'){
17      steps{
18        git branch: 'main', url: 'https://github.com/22lavanyas/Zomato-Clone.git'
19      }
20    }
21    stage("Sonarqube Analysis "){
22      steps{
23        withSonarQubeEnv('sonar-server') {
24          sh ''' $SCANNER_HOME/bin/sonar-scanner -Dsonar.projectName=zomato \
25            -Dsonar.projectKey=zomato '''
26        }
27      }
28    }
29    stage("quality gate"){
30      steps {
31        script {
32          waitForQualityGate abortPipeline: false, credentialsId: 'Sonar-token'
33        }
34      }
35    }
36    stage('Install Dependencies') {

```

```

37     steps {
38         sh "npm install"
39     }
40 }
41 }
42 }

```

Click on Build now, you will see the stage view like this

The screenshot shows the Jenkins 'Stage View' for the 'Zomato' project. The interface includes a left sidebar with navigation options like Status, Changes, Build Now, Configure, Delete Pipeline, Full Stage View, SonarQube, Rename, and Pipeline Syntax. The main area displays a 'Stage View' table with columns for various stages: Declarative Tool Install, Clean workspace, Checkout from Git, Sonarqube Analysis, Quality gate, Install Dependencies, OWASP FS SCAN, and TRIVY FS SCAN. Below this, a 'SonarQube Quality Gate' section shows the status 'Passed' for the 'zomato' project, with a 'serverside processing' status of 'Success'.

	Declarative Tool Install	Clean workspace	Checkout from Git	Sonarqube Analysis	Quality gate	Install Dependencies	OWASP FS SCAN	TRIVY FS SCAN
Average stage times: (Average full run time: ~10min 26s)	143ms	270ms	2s	22s	316ms	23s	8min 58s	37s
Dec 18 19:49	143ms	270ms	2s	22s	316ms	23s	8min 58s	37s

To see the report, you can go to Sonarqube Server and go to Projects.

The screenshot shows the 'Dependency-Check Results' page in Jenkins. It features a 'SEVERITY DISTRIBUTION' bar chart and a table of vulnerabilities. The table columns are File Name, Vulnerability, Severity, and Weakness. The results show several vulnerabilities, including CVE-2022-21222, CVE-2023-29827, CVE-2022-46175, CVE-2022-4742, CVE-2021-3803, CVE-2022-0722, CVE-2022-2216, CVE-2022-2217, CVE-2022-2218, and CVE-2022-2900.

File Name	Vulnerability	Severity	Weakness
+ css-what:3.4.2	ossindex CVE-2022-21222	High	CWE-1333
+ ejc:3.1.8	ossindex CVE-2023-29827	High	CWE-74
+ json5:1.0.1	nvd CVE-2022-46175	High	CWE-1321
+ jsonpointer:5.0.1	nvd CVE-2022-4742	Critical	CWE-1321
+ nth-check:1.0.2	nvd CVE-2021-3803	High	CWE-1333
+ parseurl:1.3.3	nvd CVE-2022-0722	High	CWE-200
+ parseurl:1.3.3	nvd CVE-2022-2216	Critical	CWE-918
+ parseurl:1.3.3	nvd CVE-2022-2217	Medium	CWE-79
+ parseurl:1.3.3	nvd CVE-2022-2218	Medium	CWE-79
+ parseurl:1.3.3	nvd CVE-2022-2900	Critical	CWE-918

You can see the report has been generated and the status shows as passed. You can see that there are 1.3k lines. To see a detailed report, you can go to issues.

Step 5 — Install OWASP Dependency Check Plugins [🔗](#)

GotoDashboard → Manage Jenkins → Plugins → OWASP Dependency-Check. Click on it and install it without restart.

Dashboard > Manage Jenkins > Tools

Docker installations

Add Docker

Docker

Name

docker

☒ Install automatically ?

Download from docker.com

Docker version ?

latest

Add Installer

First, we configured the Plugin and next, we had to configure the Tool

Goto Dashboard → Manage Jenkins → Tools →

Dashboard > Manage Jenkins > Credentials > System > Global credentials (unrestricted) > mudit097/***** (docker)

Update

Delete

Move

Update credentials

Scope ?

Global (Jenkins, nodes, items, all child items, etc)

Username ?

mudit097

☐ Treat username as secret ?

Password ?

Concealed

Change Password

ID ?

docker

Description ?

docker

Save

Click on Apply and Save here.

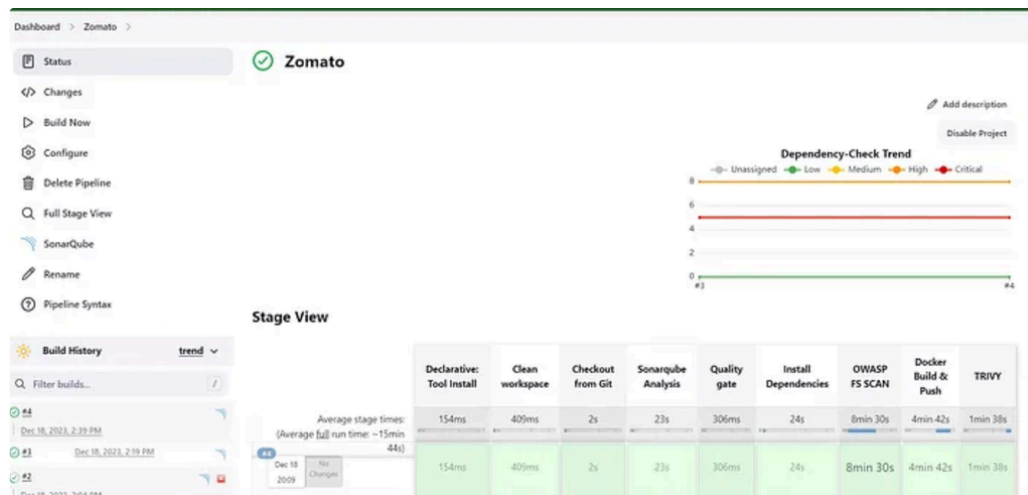
Now go configure → Pipeline and add this stage to your pipeline and build.

```

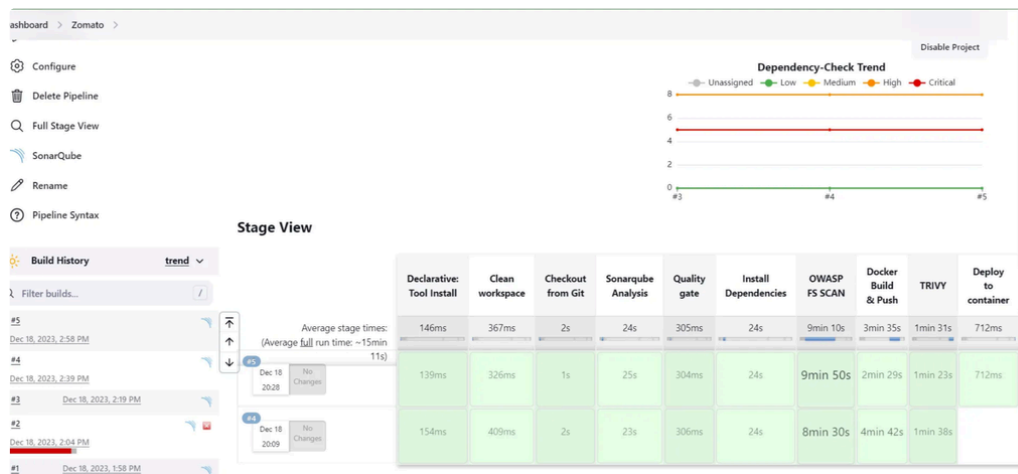
1
2     stage('OWASP FS SCAN') {
3         steps {
4             dependencyCheck additionalArguments: '--scan ./ --disableYarnAudit --disableNodeAudit',
odcInstallation: 'DP-Check'
5             dependencyCheckPublisher pattern: '**/dependency-check-report.xml'
6         }
7     }
8     stage('TRIVY FS SCAN') {
9         steps {
10             sh "trivy fs . > trivyfs.txt"
11         }
12     }

```

The stage view would look like this,



You will see that in status, a graph will also be generated and Vulnerabilities.



Step 6 — Docker Image Build and Push [🔗](#)

We need to install the Docker tool in our system, Goto Dashboard → Manage Plugins → Available plugins → Search for Docker and install these plugins

Docker

Docker Commons

Docker Pipeline

Docker API

docker-build-step

and click on install without restart

Now, goto Dashboard → Manage Jenkins → Tools →

Add DockerHub Username and Password under Global Credentials

Add this stage to Pipeline Script

```
1
2 stage("Docker Build & Push"){
3     steps{
```

```

4      script{
5          withDockerRegistry(credentialsId: 'docker', toolName: 'docker'){
6              sh "docker build -t zomato ."
7              sh "docker tag zomato mudit097/zomato:latest "
8              sh "docker push mudit097/zomato:latest "
9          }
10     }
11 }
12 }
13 stage("TRIVY"){
14     steps{
15         sh "trivy image mudit097/zomato:latest > trivy.txt"
16     }
17 }

```

You will see the output below, with a dependency trend.

When you log in to Dockerhub, you will see a new image is created

Now Run the container to see if the app coming up or not by adding the below stage

```

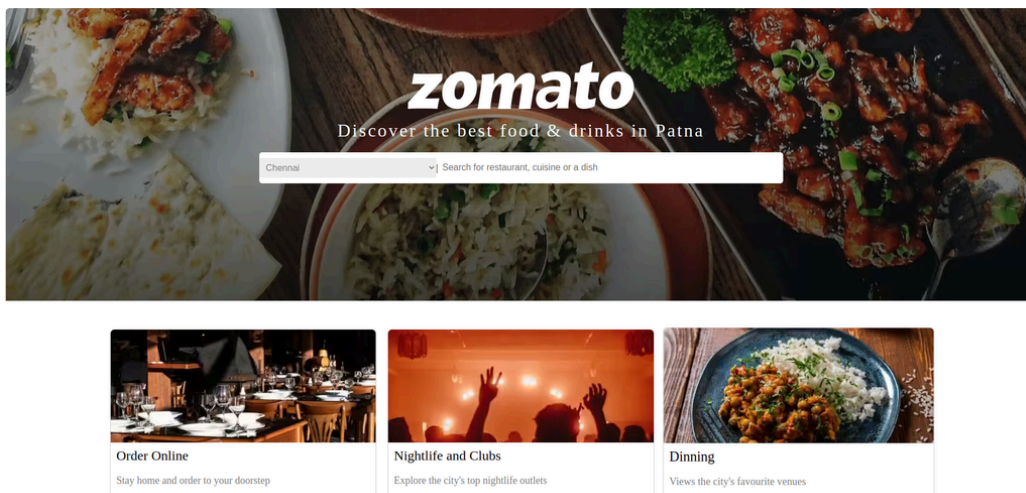
1 stage('Deploy to container'){
2     steps{
3         sh 'docker run -d --name zomato -p 3000:3000 mudit097/zomato:latest'
4     }
5 }

```

stage view

<Jenkins-public-ip:3000>

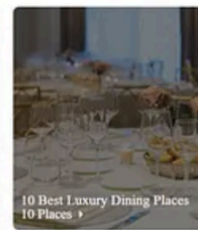
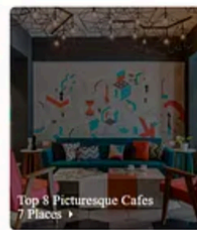
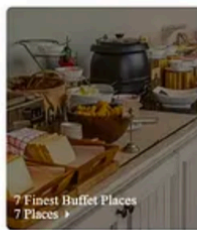
You will get this output



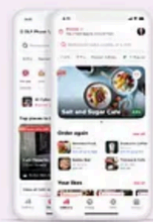
Collection

Explore curated lists of top restaurants, cafes, pubs, and bars in Ahmedabad, based on trends

[All collection in Ahmedabad](#)



Popular localities in and around Ahmedabad



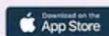
Get the Zomato app

We will send you a link, open it on your phone to download the app

☐ Email ☐ Phone

[Share App Link](#)

Download app from



Explore options near me